

# Java Parentheses Check

## Stack-Based Validation of Brackets

### 1 Objective / Aim

To write a Java program that checks whether an input string contains correctly matched and properly nested parentheses, braces, and square brackets. The program demonstrates stack-based validation using the `Deque` interface.

### 2 System Requirements

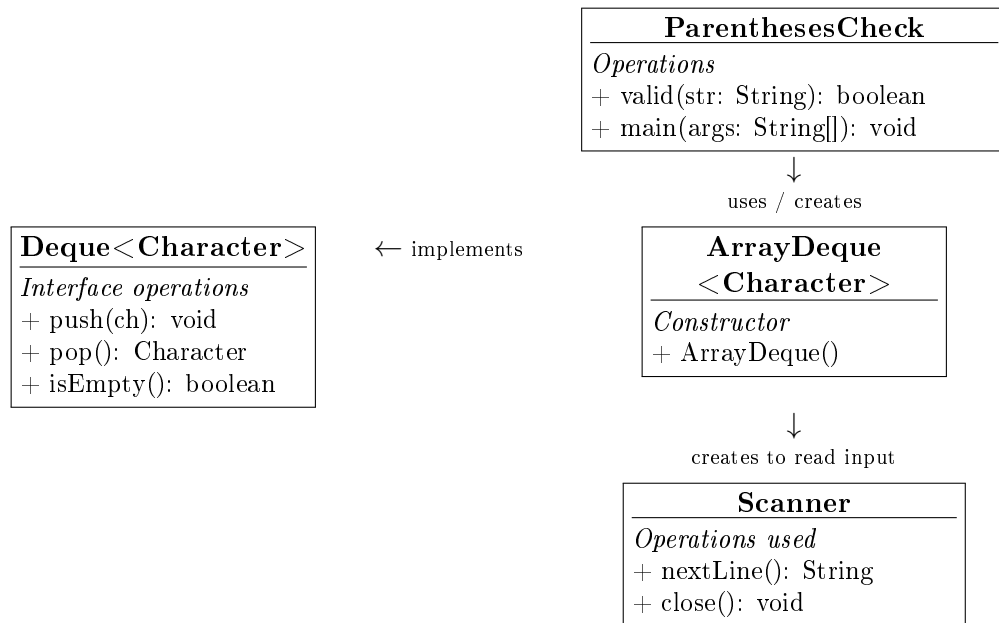
- Hardware: Standard computer capable of running a Java development environment.
- Software: Windows, Linux, or macOS; JDK 17 or later; a text editor or IDE such as VS Code, IntelliJ IDEA, Eclipse, or NetBeans.

### 3 Algorithm / Logic Description

1. Read the bracket string from standard input.
2. Create an empty `Deque<Character>` stack backed by an `ArrayDeque<Character>`.
3. Scan the string from left to right. Push each opening bracket (, {, or [ onto the stack.
4. For a closing bracket, return `false` if the stack is empty. Otherwise pop the most recent opening bracket and check that the pair matches.
5. After scanning all characters, return `true` only if the stack is empty. Print `Valid` or `Not valid`.

## 4 UML Class Diagram

The diagram uses class boxes to show the program class and the Java library types it uses. The `ArrayDeque` object is assigned to a `Deque` reference; `Scanner` reads the console input.



*UML visibility:* + public. The `ParenthesesCheck` class uses the `Deque` interface and creates an `ArrayDeque` implementation for the stack.

## 5 Source Code

Save the following source as `ParenthesesCheck.java`. The code listing has no border or line numbers.

```
import java.util.*;

public class ParenthesesCheck {

    public static boolean valid(String str) {

        Deque<Character> stack = new ArrayDeque<>();

        for (char ch : str.toCharArray()) {

            if (ch == '(' || ch == '{' || ch == '[') {

                stack.push(ch);

            }

            else {

                if (stack.isEmpty())
                    return false;

                char open = stack.pop();

                if (ch == ')' && open != '(')
                    return false;

                if (ch == '}' && open != '{')
                    return false;

                if (ch == ']' && open != '[')
                    return false;

            }

        }

        return stack.isEmpty();

    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter brackets: ");
        String input = sc.nextLine();

        if (valid(input))
            System.out.println("Valid");
        else
            System.out.println("Not valid");

        sc.close();

    }

}
```

## 6 Compilation and Execution

Open a terminal in the folder containing the source file and run:

```
javac ParenthesesCheck.java
java ParenthesesCheck
```

## 7 Sample Output

For the input `{[()]}`, one run appears as:

```
Enter brackets: {[()]}
Valid
```

For mismatched, unclosed, or prematurely closed brackets, the program prints `Not valid`.

## 8 Result

The program validates properly nested bracket pairs using a last-in, first-out stack. The final empty-stack check detects opening brackets that were never closed.